Generating Loop Termination Proofs for eBPF in LLVM

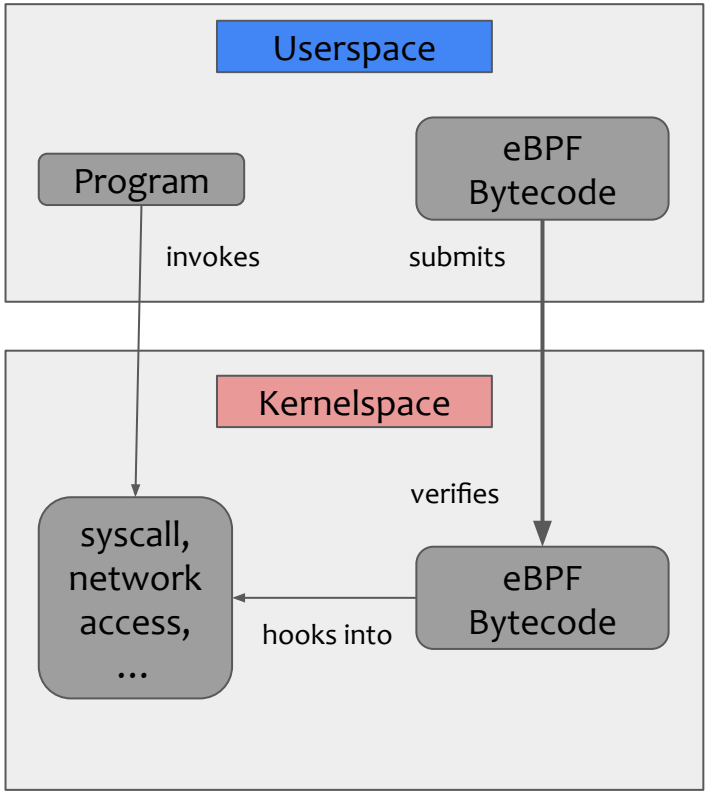
Deniz Mersinlioğlu
Advisor: Martin Fink
Chair of Computer Systems
https://dse.in.tum.de/



19.05.2026 – 21.09.2026

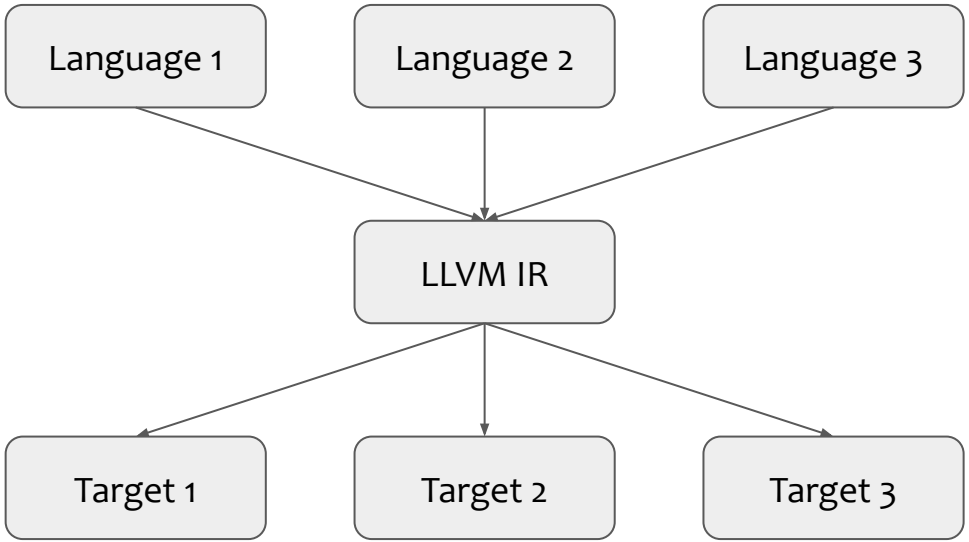
What is eBPF?

- An ISA, running in a simulated CPU
- Untrusted user programs in kernelspace
- JIT Support
- Better performance
  - No mode switching
- **Verification of programs required**



What is LLVM?

- Compiler infrastructure
- Separate Frontends and Backends
- Common Analysis



Outline

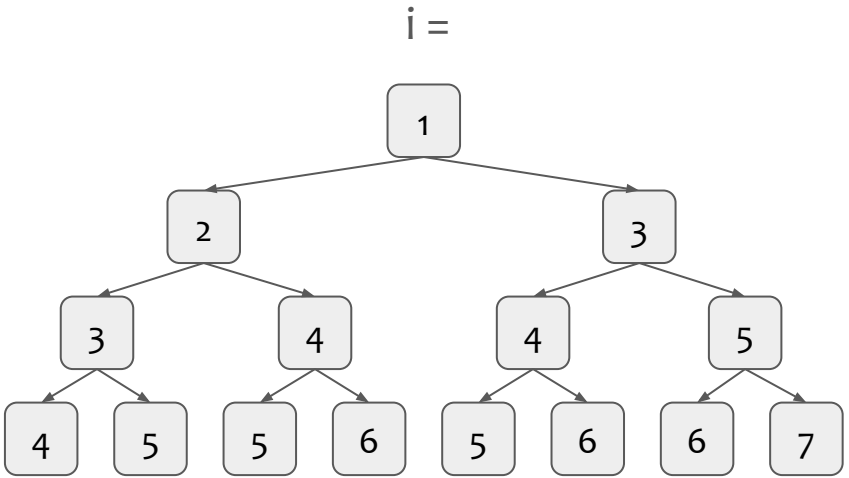
- ~~Background~~
- Motivation
- Design
- Implementation
- Evaluation

State-of-the-art

- Loop termination verified via full state exploration
- Maximum verification cost constraint

```
extern unsigned int random();
extern void side_effect(int i);
int hello_world(void *ctx) {
    for (int i=0;i<5000;) {
      if (random() & 0x1) {
        i += 1;
      } else {
        i += 2;
      }
      side_effect(i);
    }

    return 0;
}
```



Research gap

False Negatives

As seen in previous slide: “brute force” on program states sometimes inflates costs

Verifier reasons
“from scratch”

The verifier only sees the final binary. No awareness of high level concepts such as “loop variables”, although these are widespread in practice.

Problem statement

Can we verifiably prove at compile time that all loops in a given eBPF program are guaranteed to terminate?

Motivation

- Limitations of current verifier restrict potential use cases
- The compiler “understands” loops
- Get this information to the verifier
- Verify more loops
- **If all loops terminate: Program terminates**

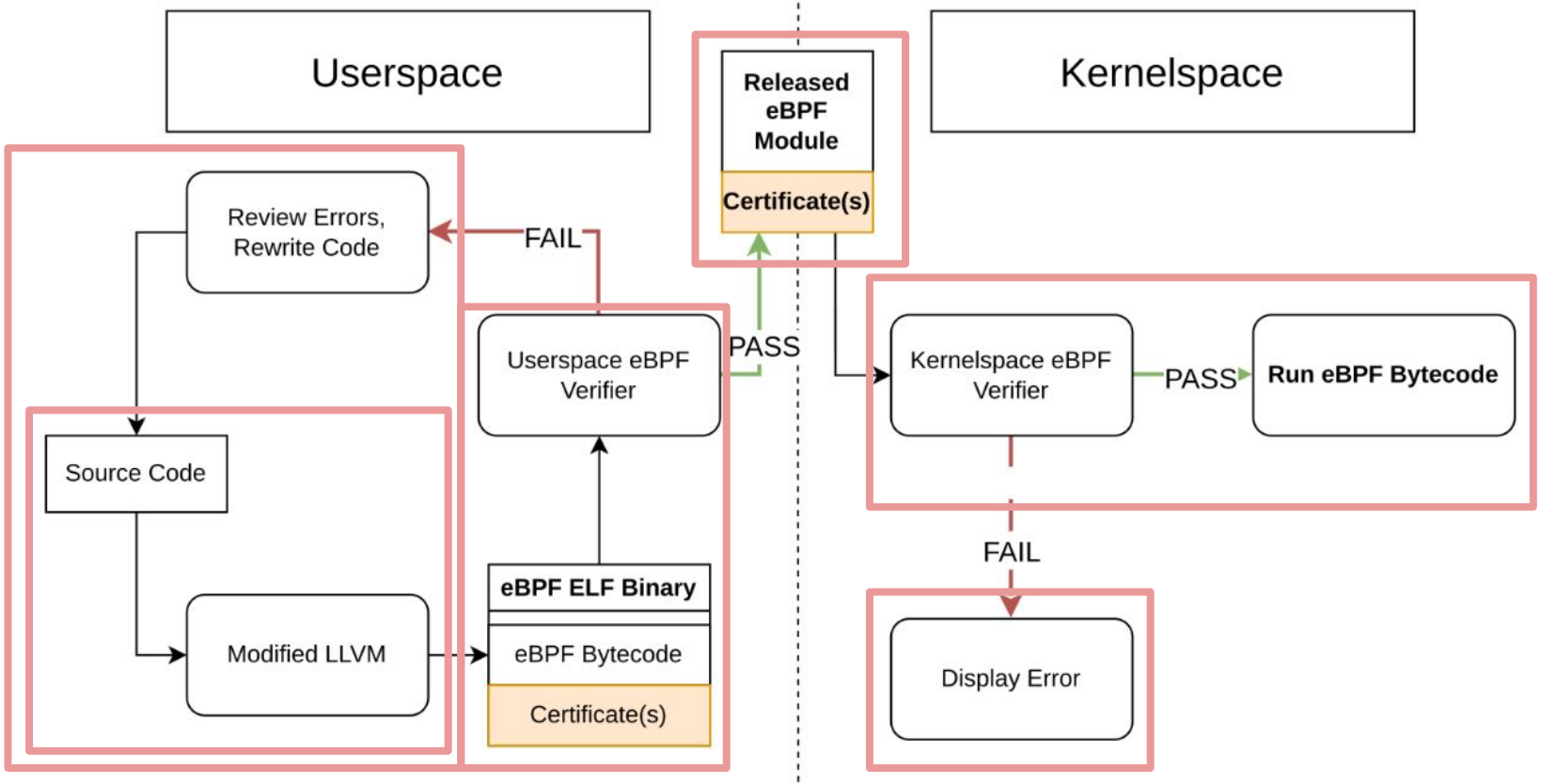
Key Idea

- We check / prove loop termination in the compiler
  - The compiler auto-detects loops and finds the loop variable and bounds if available
- The resulting certificate references values and instructions inside the program
  - For example, the loop variable, bound values determined at runtime, backedges…
- Certificate + Program later verified together

Outline

- ~~Background~~
- ~~Motivation~~
- Design
- Implementation
- Evaluation

Design - Overview



System: A proof generator for loop termination

Overall structure:

- Augment compiler, use its loop analysis
- Attach “loop measures” to program
  - One loop measure per loop
- Emit certificate in resulting binary

Design - Loop Measures

- Expression we can evaluate directly before the start of the loop
  - Inspired by W-Functions [1]
- Non-negative integers
- Strictly descending between loop executions
  - Every backedge decrements it overall
- Finite
- If a loop measure fulfilling these properties exists: **The loop cannot be infinite**

[1]: R. W. Floyd. “Assigning Meanings to Programs.” In: Program Verification: Fundamental Issues in Computer Science. Ed. by T. R. Colburn, J. H. Fetzer, and T. L. Rankin. Dordrecht: Springer Netherlands, 1993, pp. 65–81. isbn: 978-94-011-1793-7. doi: 10.1007/978-94-011-1793-7_4

Design - Loop Termination Proofs

Program

```
…

for (int i=0; i<500; i++)
{

      …

}

…
```

Termination

```
…

Start: 0, End: 500,
Stride: 1

i ∈ [0, 500) ascending

→ 500 - i ∈ (0, 500]
descending, (0, 500] ⊆ ℕ0
```

Certificate

```
…

Measure: 500 - i
Var i in Register (reg)
Backedges: (...)

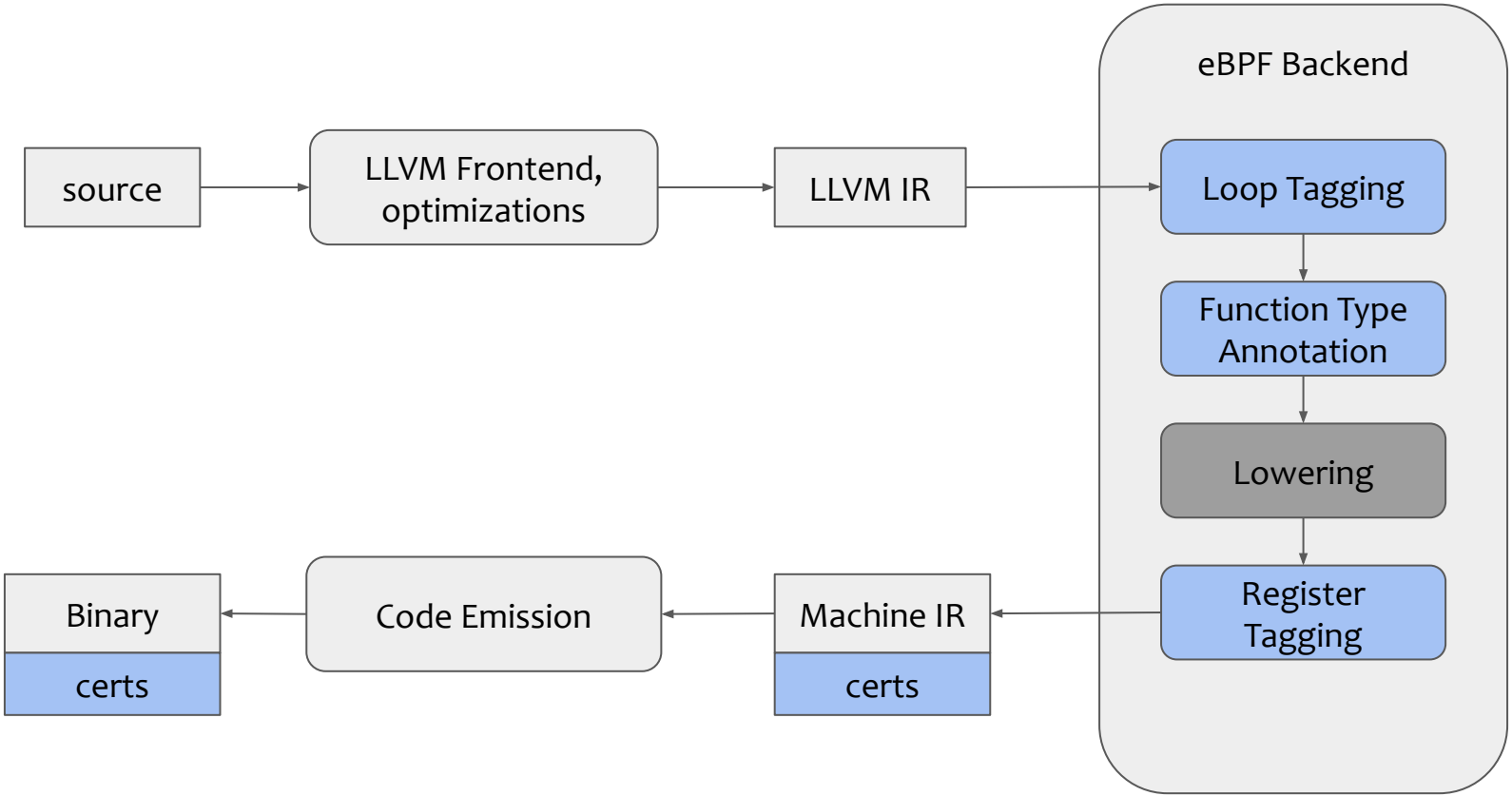
…
```

For each loop, we attempt to generate a loop measure

Outline

- ~~Background~~
- ~~Motivation~~
- ~~Design~~
- Implementation
- Evaluation

Implementation overview



Outline

- ~~Background~~
- ~~Motivation~~
- ~~Design~~
- ~~Implementation~~
- Evaluation

Evaluation

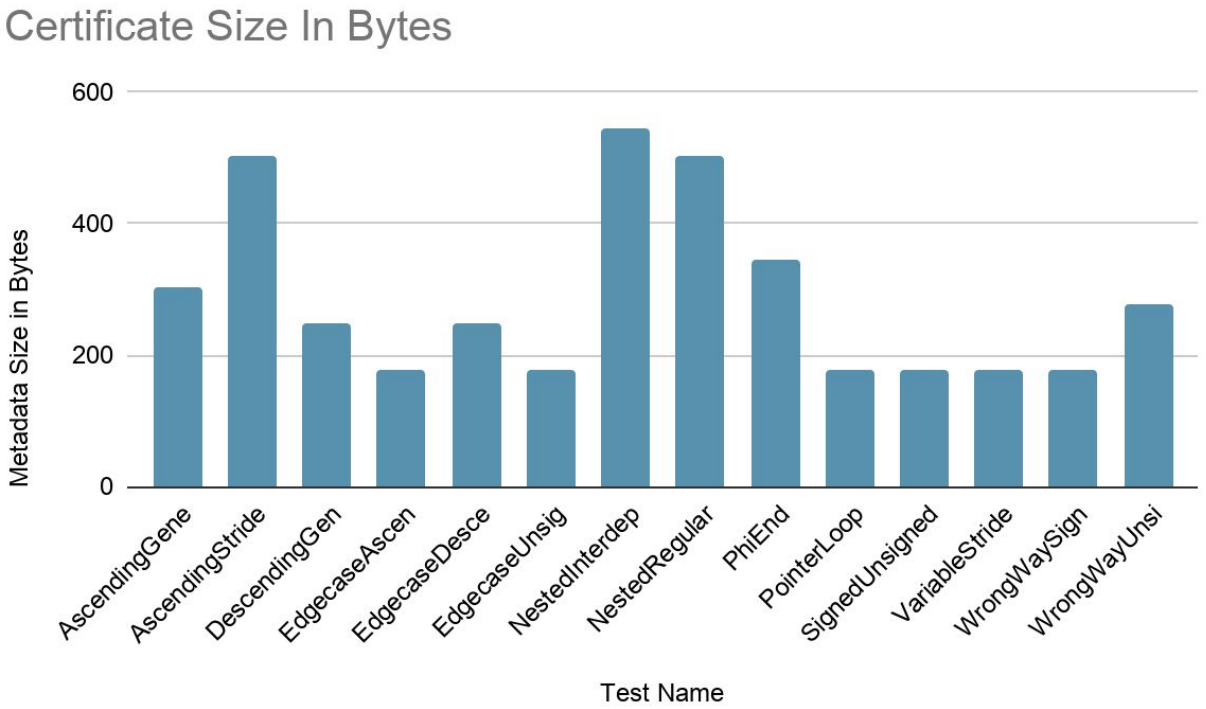
- Unit tests: What kinds of loops can our suite handle?
  - Nested loops, signed and unsigned overflows, pointer arithmetic, …
- Size footprint: What is the size overhead of our loop certificates?

Evaluation - Test Cases

- Simple loops valid, even very long ones
- LLVM itself detects many infinite loops

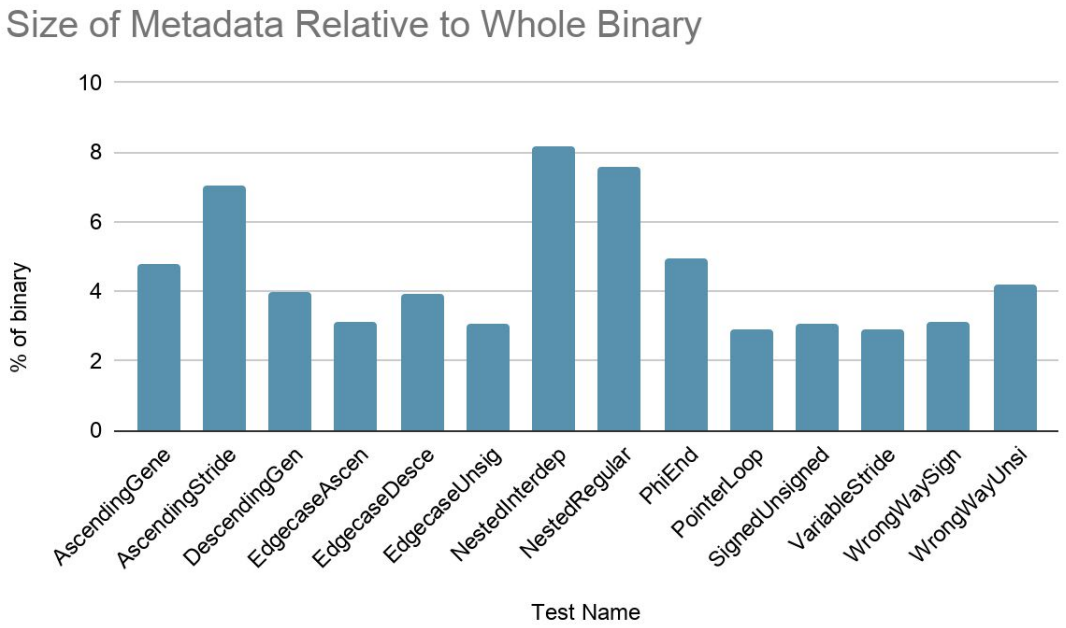
- Nested loops currently can't be verified
  - Theoretically possible
- Cases where the loop stride or endpoint are not invariant can't be verified
  - Due to LLVM's own loop analysis infrastructure's limitations
- Real-life projects: Linking / compilation remains buggy

Evaluation - Size Footprints



Roughly 100 - 300 bytes per loop!

Evaluation - Size Footprints



Approx. 3-5% of the resulting binary

Summary

The current eBPF verifier's ability to verify loop termination is limited

- Loop verification via "manual" brute-force checking
- Many loops wrongly fail to verify

Our implementation uses LLVM's loop analysis to generate termination certificates

- Compiler insights on high level loop structure can be leveraged
- We can decouple verification time from execution time
- We lay the groundwork for verifying many more types of loops
- Outlook: **Prove full program safety, not just one narrow aspect!**

Backup